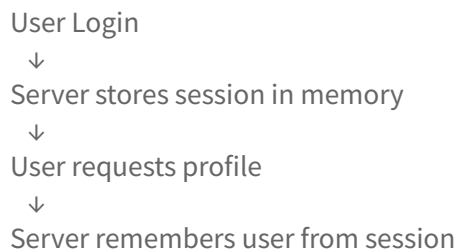


Fundamentals of Backend Architecture,

1. Stateful Servers!

A stateful server **remembers information** about a user **between requests**. The Server **stores session data in RAM**.



Advantages

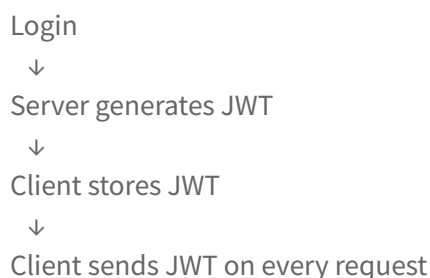
- Easy implementation
- Traditional web applications

Disadvantages

- Difficult to scale
- If server crashes, session may be lost
- Load balancer must send user to same server

2. Stateless Servers!

A stateful server **stores no information** about a user **between requests**. Each request contains everything needed..



Advantages

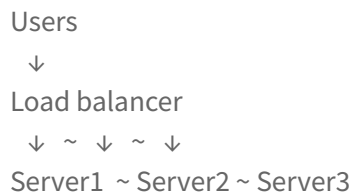
- Easy horizontal scaling
- Better for mobile apps
- Cloud-friendly

Disadvantages

- Token management required

3. Load Balancing!

When one **Server cannot handle all traffic**, **requests** are **distributed** across **multiple servers**.



Responsibilities

- Distribute traffic
- Health checks
- Failover
- SSL termination

Popular load balancers

- Nginx
- HAProxy
- Traefik
- Envoy

4. Nginx Example!

Nginx is commonly used as: **Reverse proxy**, **Load balancer**, **Static file server**.



Example:

```
Upstream backend {  
    Server 10.0.0.1:3000;  
    Server 10.0.0.2:3000;  
    Server 10.0.0.3:3000;  
}  
Server {  
    Listen 80;  
    Location / {  
        Proxy_pass http://backend;  
    }  
}
```

5. Different Types of Scaling!

1. Vertical Scaling: **Increase power** of one **server**.

4 CPU to 16 CPU || 8gb RAM to 16gb RAM

Advantages

- Easy

Disadvantages

- Expensive
- Hardware limits

2. Horizontal Scaling: **Add** more **servers**.

Users

↓

Load Balancer

↓

Multiple Servers

Advantages

- Highly scalable
- Fault tolerant

6. Microservices Architecture!

Instead of one huge Application - Split into services.

Monolith

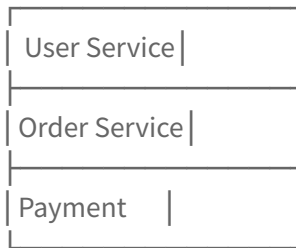
User Service | Order Service | Payment Service | Notification Service

Architecture:

Mobile App



API Gateway



Advantages

- Independent deployment
- Team ownership
- Better scalability

Disadvantages

- Complexity
- Network communication
- Monitoring challenges

7. Authentication!

Authentication: Who you are. Authorization: What can you do?

Login



Verify credentials



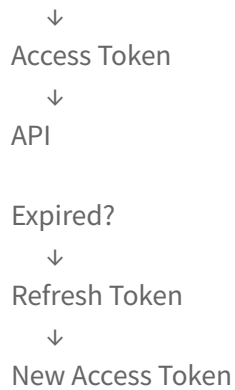
Issue Access Token



Access protected APIs

Modern auth stack:

Mobile App

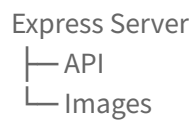


8. File Serving!

File can be: Images, Videos, PDFs, Documents

Bad Approach:

Store files inside API server.



Happy traffic can overload API server

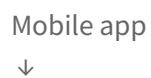
Better Approach:



Example:

Amazon Web Services S3
Cloudflare R2
Google cloud Storage

Typical upload flow:



backend
↓
Generate Upload URL
↓
Direct Upload to Storage

9. Event Broker!

An event broker allows services to communicate asynchronously

Without broker:

Order Service
↓
Notification Service

If notification fail => Order may fail

With broker:

Order service
↓
Event Broker
↓
Notification Service

Popular brokers:

RabbitMQ
Apache Kafka
Redis Pub/Sub

Example:

Order Created
↓
Kafka Topic
↓
Email service | Analytics service | Inventory service

10. Caching and CDNs!

Caching: Store frequently accessed data.

Without cache:

Request



Database

Every request hits DB

With cache:

Request



Cache



Database (if miss)

Popular cache:

Redis

CDN: stores files near users globally

Origin server



CDN



Users Worldwide

Benefits:

Faster images

Lower server load

Global performance

Popular CDNs:

Cloudflare

Amazon Web Services

Fastly

11. Rate Limiting!

Prevents abuse and protects servers

Example:

100 request per minute = 429 too many requests

12. How everything fits together!

A modern production backend often looks like this:

